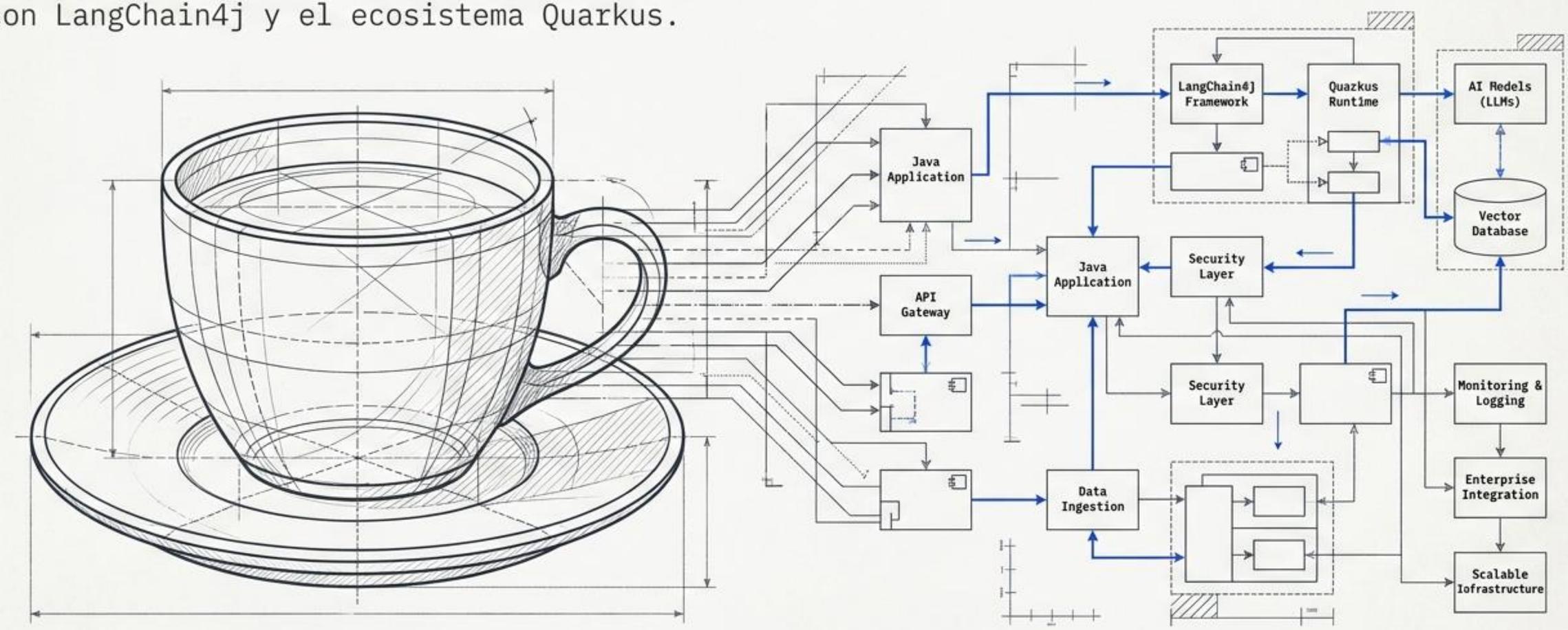
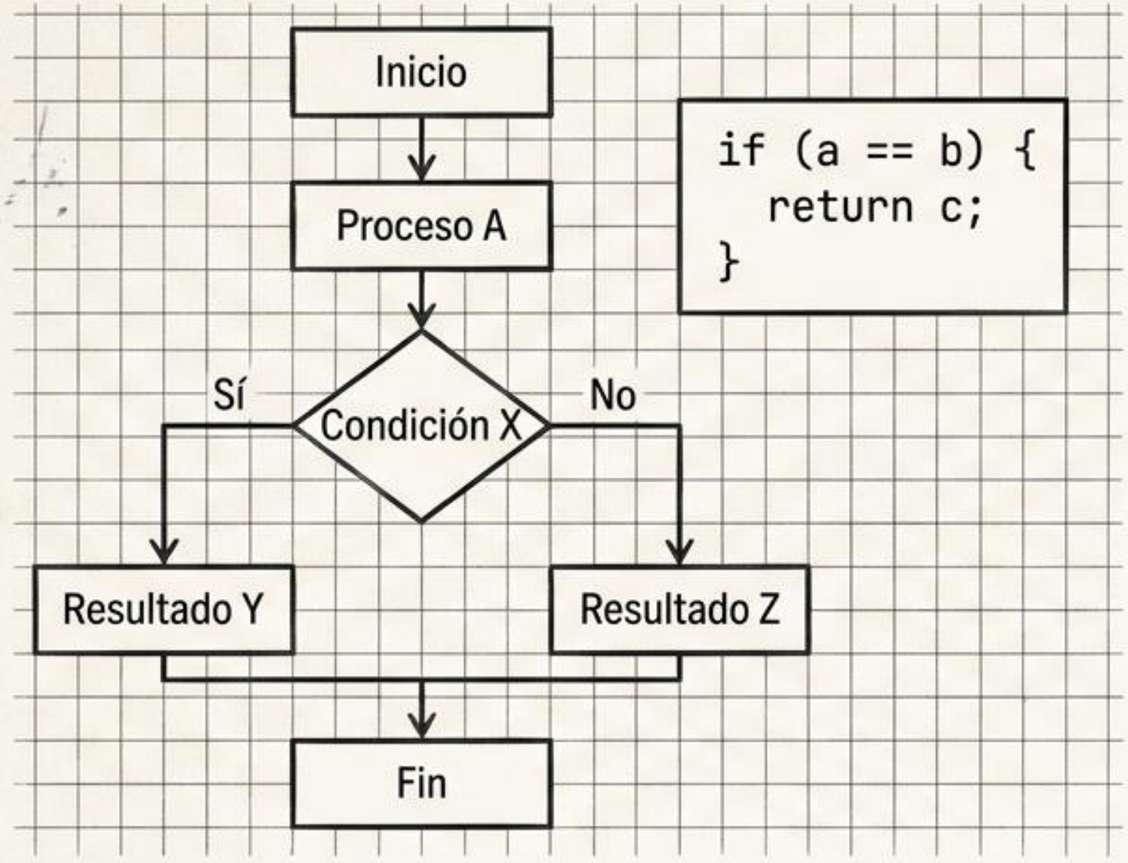
De Hello World a Producción Empresarial: Ingeniería de IA en Java

Construyendo sistemas robustos, seguros y escalables
con LangChain4j y el ecosistema Quarkus.



NotebookLM

Mundo Determinista




```
if (a == b) {
  return c;
}
```

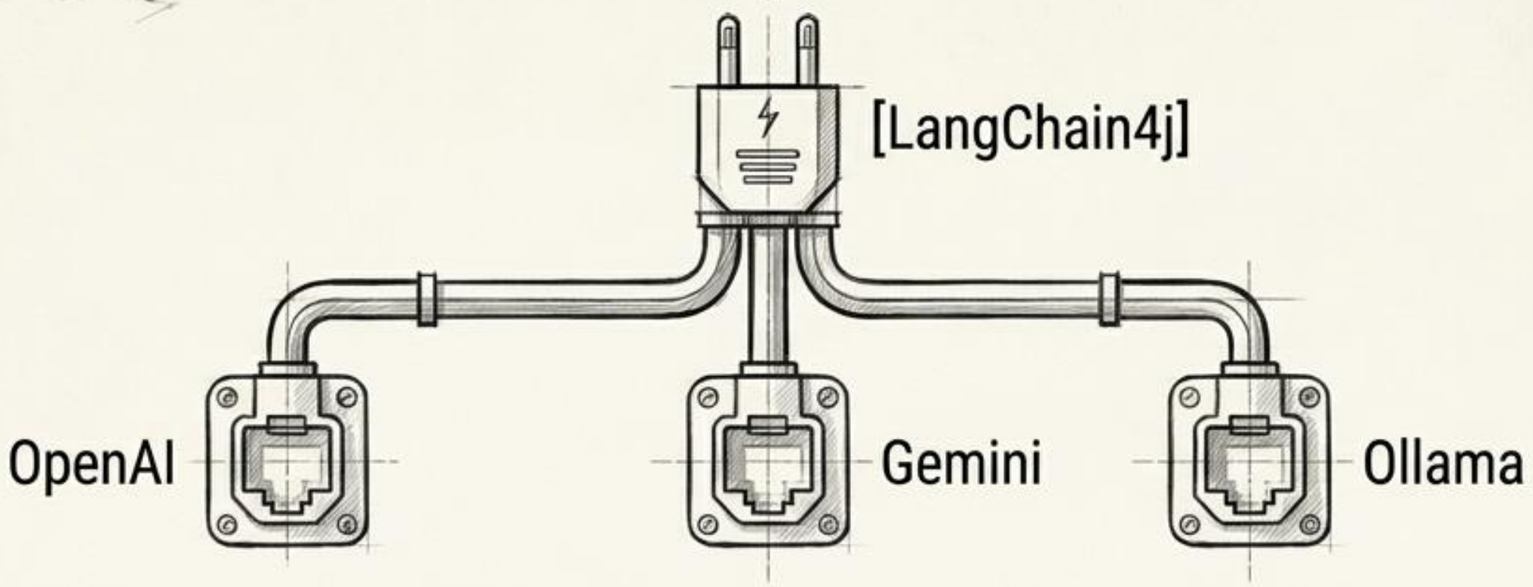
- 100% Predecible
- Reglas Estrictas
- Estado Explícito

Mundo Probabilístico

- No Determinista
- Propenso a Alucinaciones
- Sin Estado (Stateless)

LangChain4j es la capa de abstracción que impone el rigor de la ingeniería de software sobre el caos de los modelos probabilísticos.

NotebookLM



Dimensión	Modelos Cloud (OpenAI, Gemini)	Modelos Locales (Ollama, JLama)
Privacidad	Baja (Datos salen de la red)	Alta (Aislamiento total)
Costo	Por Token (Gasto variable continuo)	Hardware y Energía (Costo fijo)
Latencia	Red (Dependiente de API)	CPU/GPU Local (Hardware propio)

LangChain4j permite cambiar de proveedor modificando una sola variable de configuración, evitando el vendor lock-in.

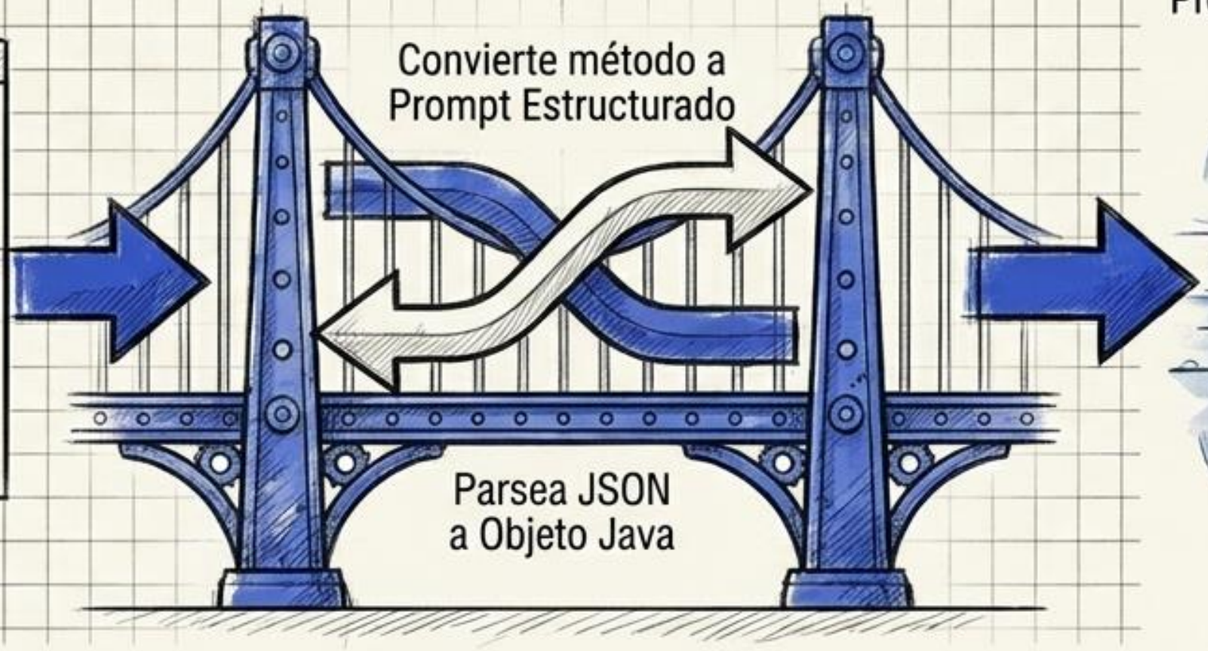
NotebookLM

AI Services: El puente de traducción de String a POJO

Mundo Java

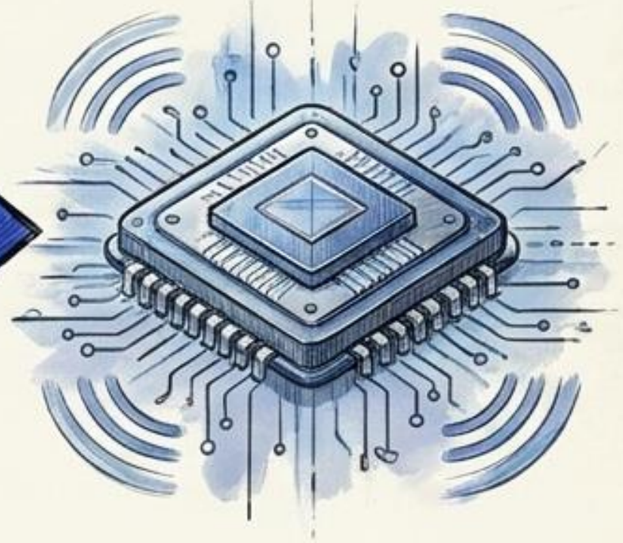
```
@SystemMessage
@UserMessage
Persona extraerDatos(String texto);
```

Motor LangChain4j



Modelo de IA

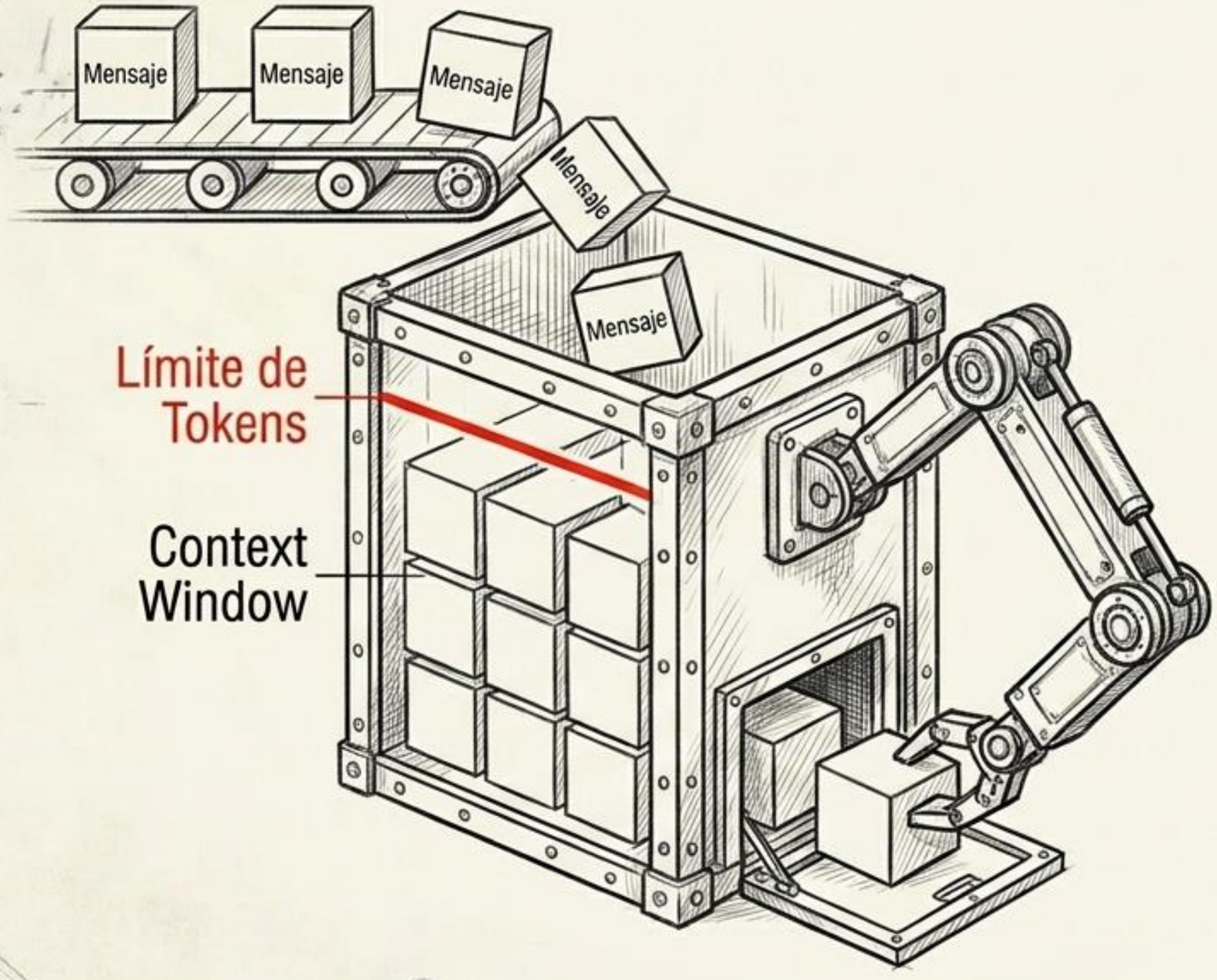
Procesamiento de Lenguaje Natural



En lugar de **parsear Strings manualmente**, definimos **QUÉ** queremos usando **interfaces Java**. El modelo devuelve objetos de dominio (POJOs) estrictamente tipados.

NotebookLM

Problema 1: Amnesia -> Ingeniería de Memoria



El Problema

Los LLMs son “Stateless” (sin estado).

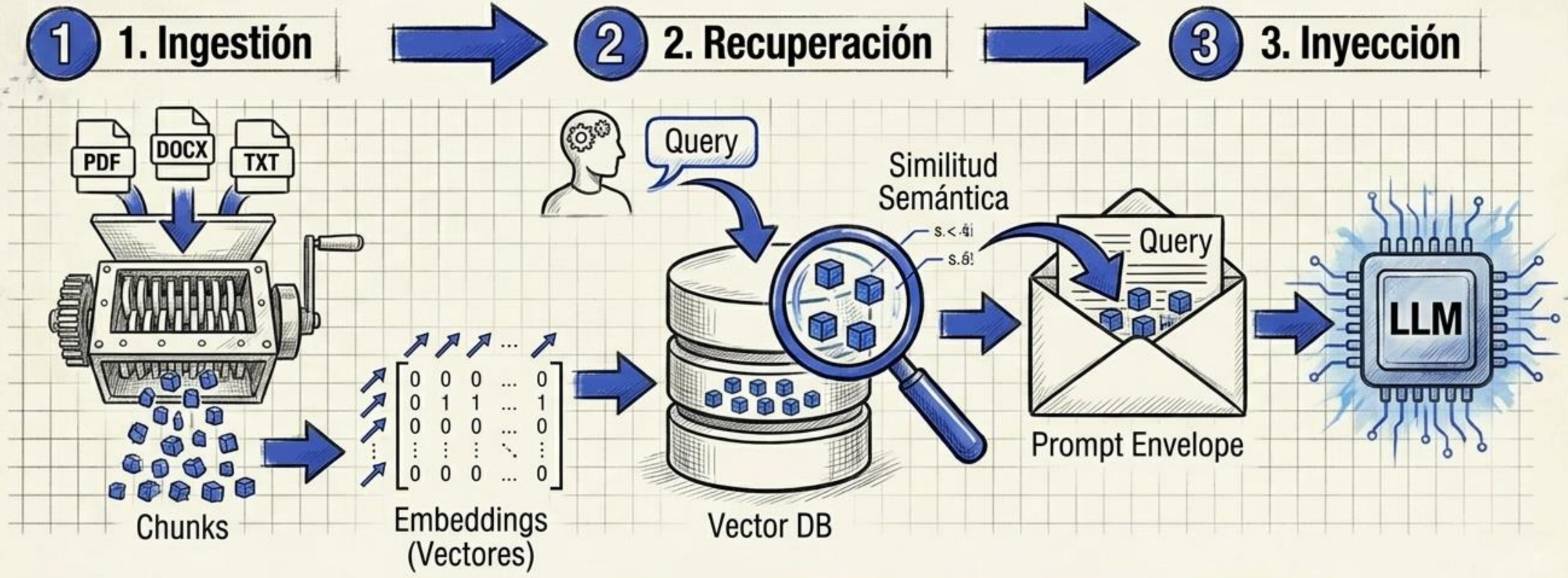
Enviar todo el historial en cada petición agota la ventana de contexto y dispara el costo.

La Solución (Eviction)

A	**`MessageWindowChatMemory`** (Descarta por cantidad de interacciones)
B	**`TokenWindowChatMemory`** (Calcula costo real y descarta por volumen de tokens)

NotebookLM

Problema 2: Alucinaciones -> RAG (Retrieval-Augmented Generation)



RAG transforma a un LLM: de un escritor creativo propenso a inventar, a un investigador preciso y fundamentado exclusivamente en tus datos.

NotebookLM

Problema 3: Aislamiento -> Herramientas y Protocolos

@Tool (Function Calling)

MCP (Model Context Protocol)

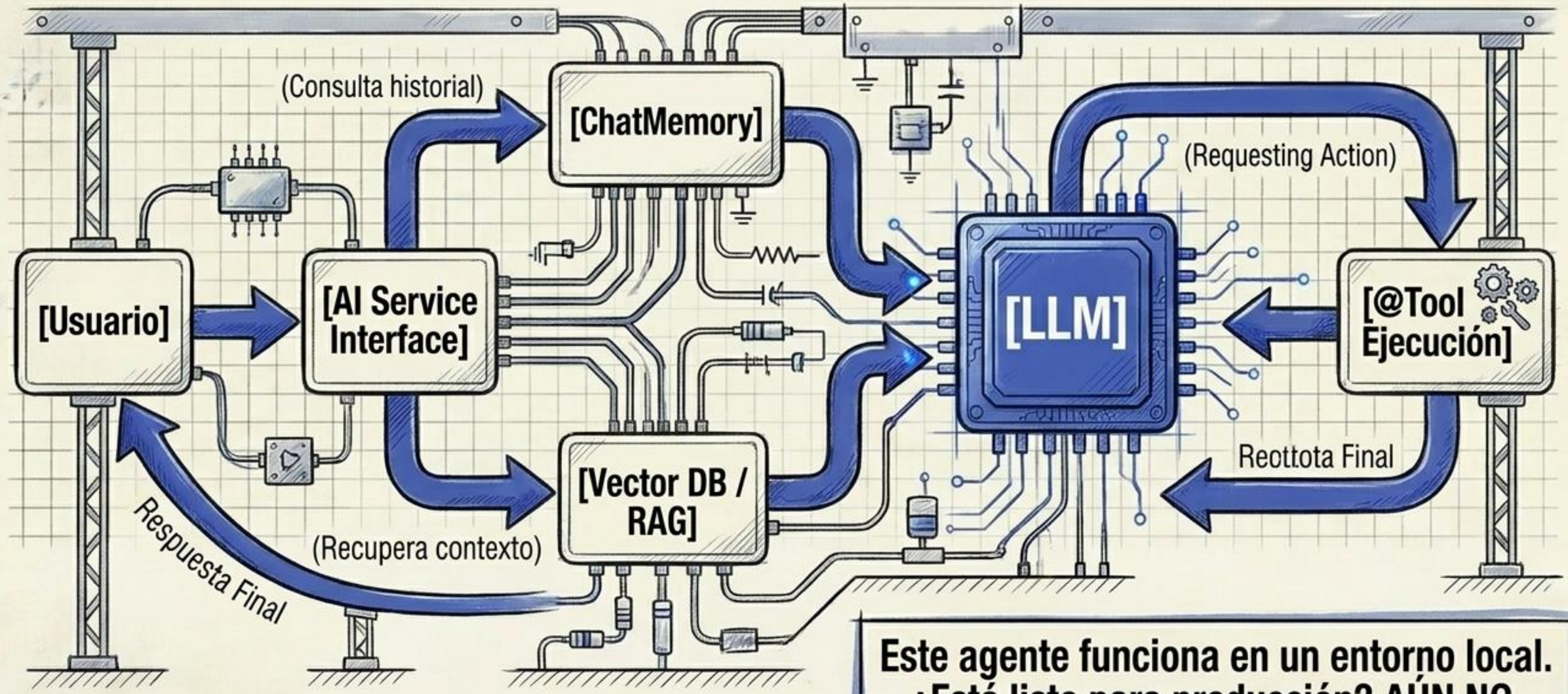
Ejecuta métodos Java locales
(Ej. Consultar PostgreSQL, enviar email)

Estándar abierto para conectar IA directamente con servidores y APIs externas sin código puente complejo.

Advertencia: Otorgar 'manos' al modelo introduce riesgos inmensos de seguridad y requiere supervisión estricta.

NotebookLM

Síntesis Arquitectónica: Anatomía del Agente Aumentado



Este agente funciona en un entorno local.
¿Está listo para producción? AÚN NO.

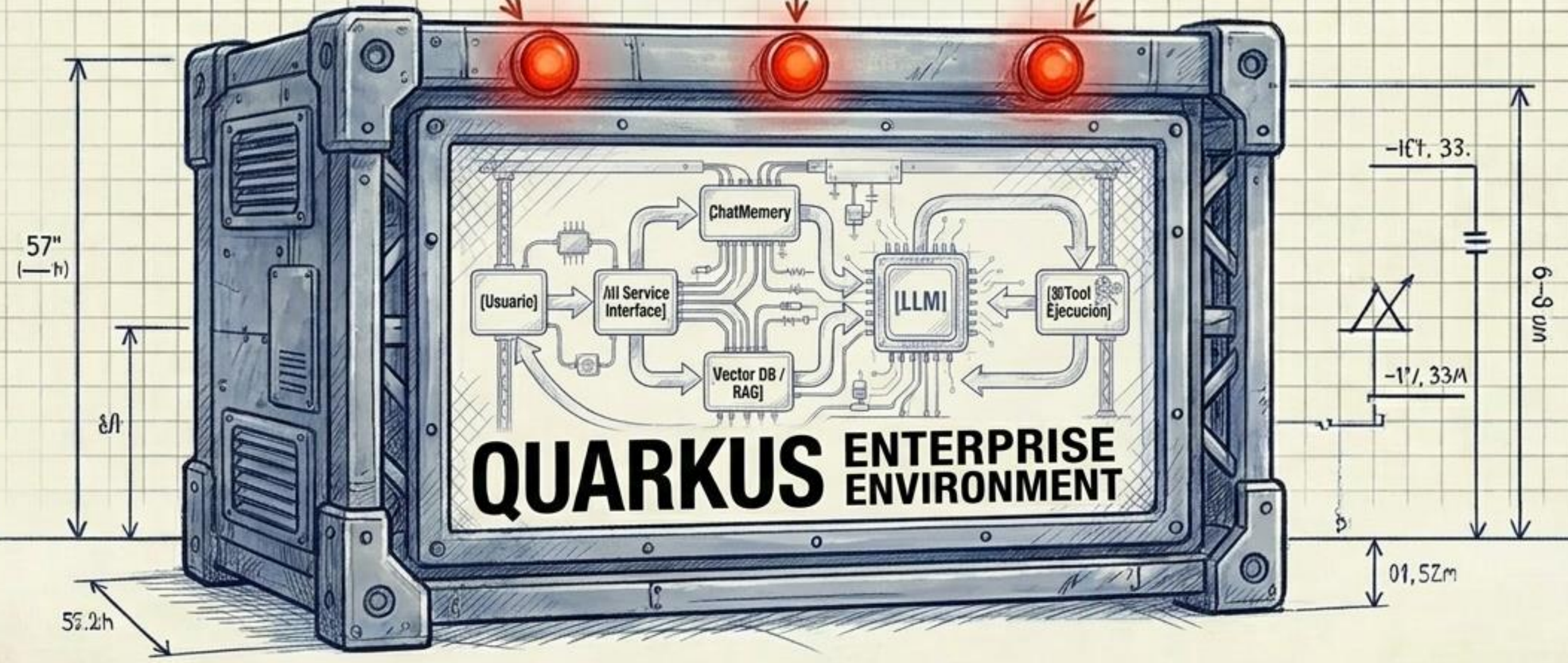
NotebookLM

El abismo de la producción: Prototipear es fácil, escalar es difícil

Riesgo de Seguridad
(Inyecciones de Prompt, Fuga de datos)

Caos No Determinista
(Fallos en tests, regresiones de código)

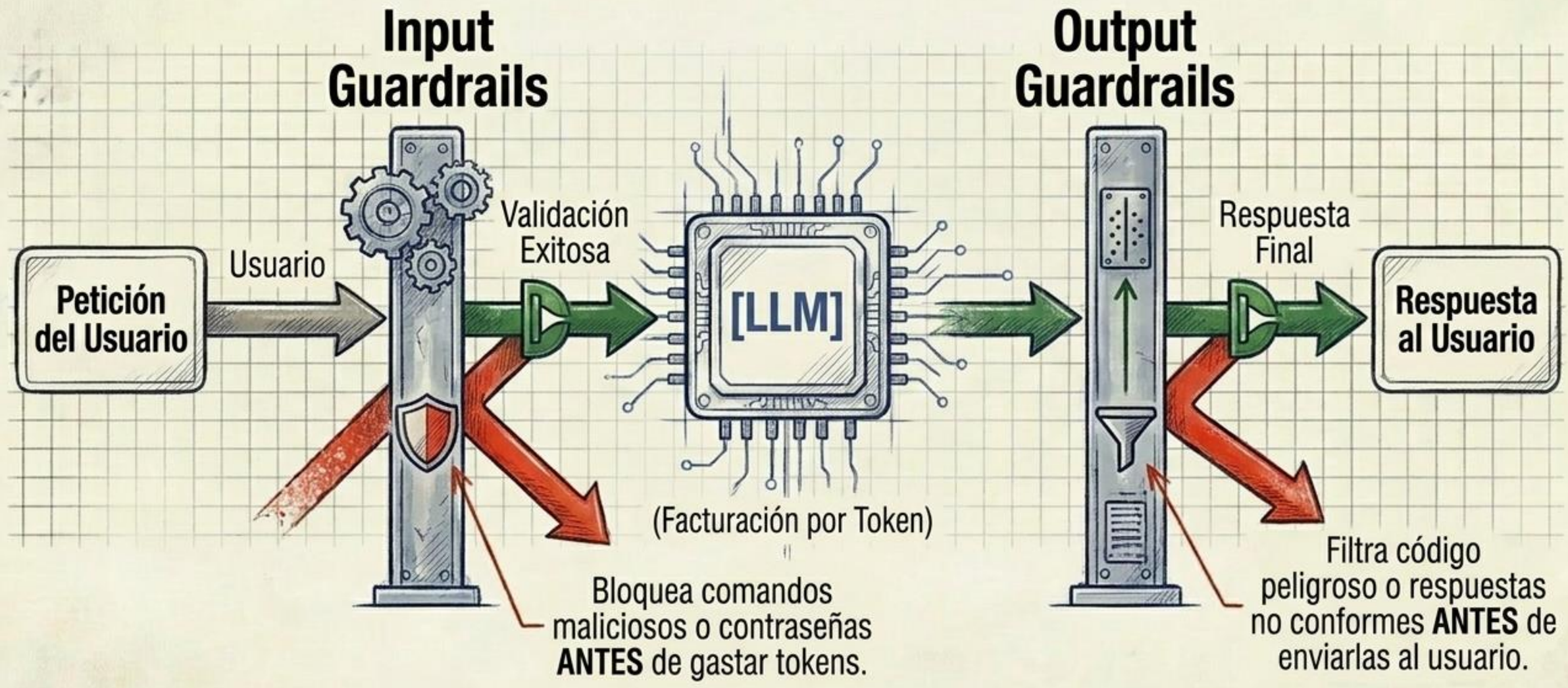
Pérdida de Control de Costos
(Consumo infinito de tokens)



Para sobrevivir en producción, necesitamos los escudos defensivos del ecosistema Quarkus.

NotebookLM

Defensa Nivel 1: Guardrails de Seguridad (Fail-Fast)



NotebookLM

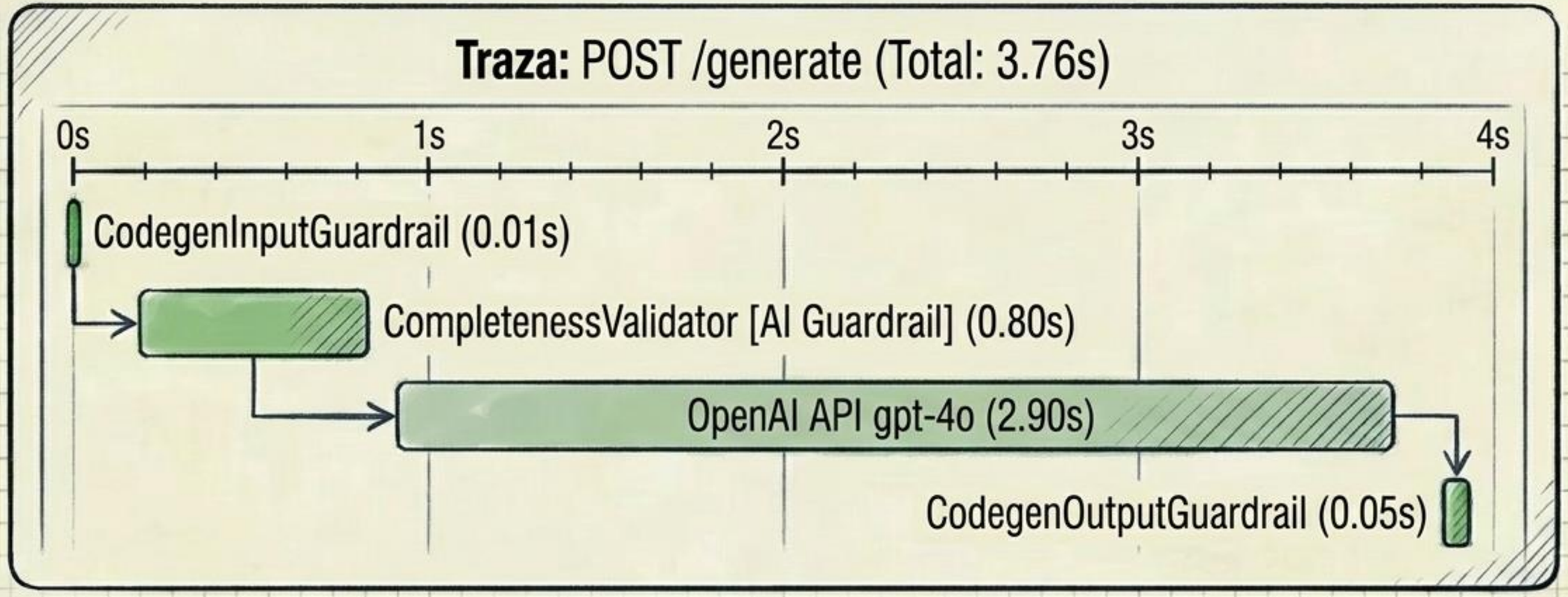
Defensa Nivel 2: Testing automatizado para lo impredecible

El código no determinista requiere evaluaciones difusas (fuzzy testing), no aserciones exactas.

Estrategia de Testing	Mecanismo	Uso Ideal
Similitud Semántica	Compara vectores. Verifica si la respuesta es > 85% similar al texto esperado.	Evaluaciones rápidas y económicas.
AI Judge (LLM evalúa LLM)	Un segundo modelo evalúa la lógica, el tono y la corrección estructural.	Validaciones lógicas complejas. Cuesta tokens.
Algoritmos Puros	Cálculo matemático (ej. Levenshtein) para medir la distancia entre cadenas.	Validación determinista sin red ni IA.

NotebookLM

Defensa Nivel 3: Observabilidad y Trazabilidad



Con OpenTelemetry nativo en Quarkus, rastreamos exactamente dónde se consume el tiempo. La caja negra de la IA se vuelve transparente.

NotebookLM

Control de Costos (FinOps): El código ineficiente drena presupuesto

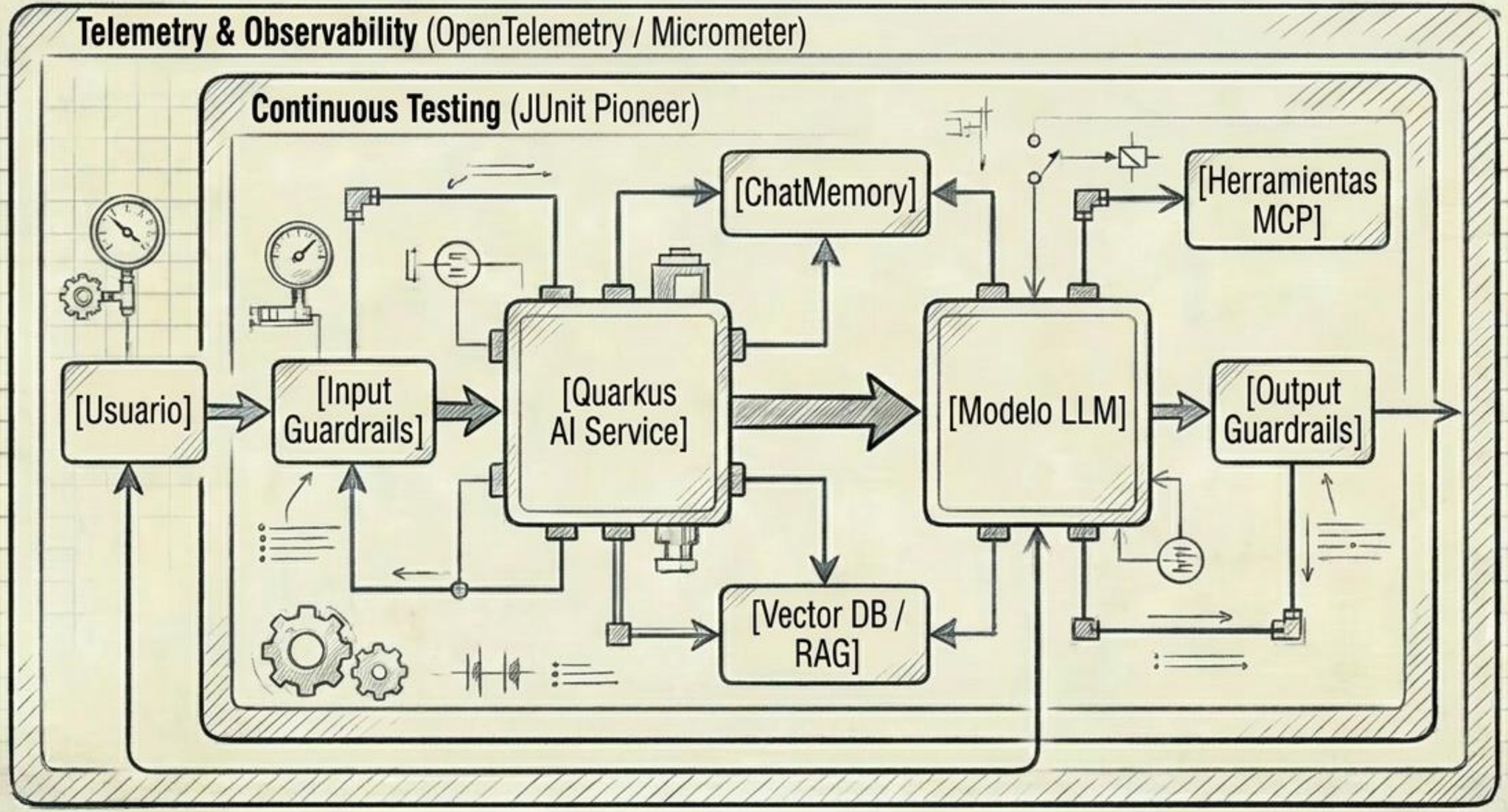
Input:
$5.00 / 1M Tokens

Output:
$20.00 / 1M Tokens

En la ingeniería tradicional, un bucle infinito colapsa el servidor.
En la IA, un bucle infinito agota la tarjeta de crédito de la empresa.

NotebookLM

El **Blueprint Definitivo:** Arquitectura de IA en Producción



NotebookLM

“El futuro del software no es escribir menos código, es orquestar inteligencia con rigor.

1. Construye por capas de abstracción.
2. Protege los inputs para ahorrar dinero; protege los outputs para evitar desastres.
3. Lo que no se puede testear y monitorear, no debe ir a producción.

Documentación
Quarkus
LangChain4j

NotebookLM